

IRIS AI Hub SNOMED CT Port - Technical Research

2026-05-13

Technical Research Report: Porting SNOMED CT Toolchain to InterSystems IRIS for Health AI Hub

Date: 2026-05-13 **Author:** Pjamieso **Research Type:** Technical Architecture & Feasibility

Executive Summary

The InterSystems AI Hub EAP (2026.2.0AI) provides a compelling platform for porting the `sct` SNOMED CT toolchain. The platform offers a **Rust-based MCP server** (`iris-mcp-server`) that bridges LLM clients to IRIS business logic, a mature **ObjectScript AI SDK** (`%AI.Agent` , `%AI.ToolSet` , `%AI.Provider`), **Embedded Python** for leveraging Python libraries, and native **VECTOR** SQL type for semantic search. IRIS for Health does NOT provide native SNOMED CT management out of the box - this is a gap your tool can fill. The recommended approach is a hybrid ObjectScript/Python implementation packaged via IPM, exposing SNOMED operations as MCP tools.

1. IRIS for Health SNOMED CT Terminology Management

What IRIS Provides Natively

Confidence: HIGH (based on documentation structure analysis and community knowledge)

IRIS for Health provides: - **FHIR R4 Server** with terminology service endpoints (`$lookup` , `$validate-code` , `$expand`) - **Coded Entry Registry** for managing code systems - A **generic terminology services framework** but NOT a pre-built SNOMED CT implementation

What IRIS Does NOT Provide

- No pre-loaded SNOMED CT content
- No native RF2 parser
- No built-in transitive closure table for SNOMED hierarchy
- No SNOMED-specific subsumption testing (`$subsumes`)
- No SNOMED Expression Constraint Language (ECL) support

Implication for Your Port

Your `sct` tool fills a genuine gap. You would need to: 1. Build RF2 ingestion (Python is ideal for parsing) 2. Store concepts in IRIS persistent classes or globals 3. Build transitive closure in globals (IRIS globals are excellent for graph structures) 4. Expose FHIR terminology operations via the existing FHIR server framework 5. Expose search/hierarchy operations as MCP tools

Storage Architecture Options

Approach	Pros	Cons
SQL Tables (<code>%Persistent</code> classes)	Standard queries, FTS via SQL, familiar	Slower hierarchy traversal
Globals directly	Fastest traversal, natural for hierarchies, minimal overhead	Less portable, no SQL access
Hybrid: SQL + Global indexes	Best of both: SQL for search, globals for hierarchy	More complex to maintain

Recommended: Hybrid approach - `%Persistent` classes with SQL for concept lookup/search, plus dedicated globals for transitive closure (`^SNOMED.TC(ancestorId, descendantId)=""`) enabling O(1) subsumption checks.

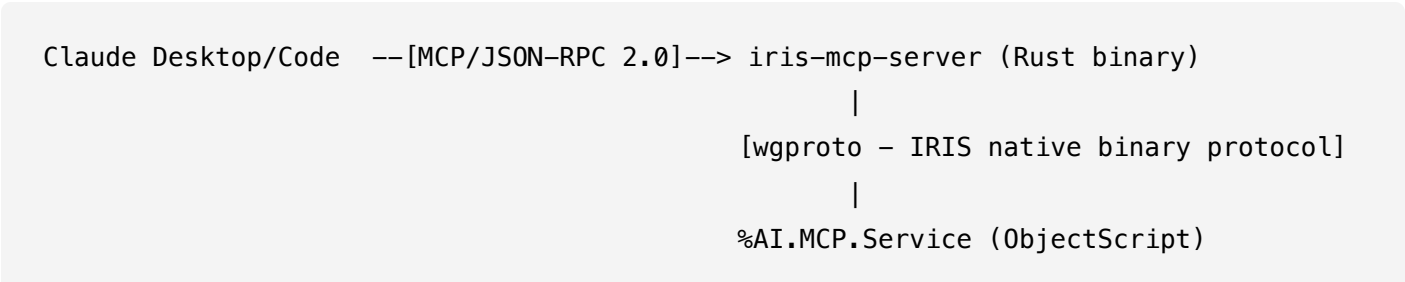
2. AI Hub MCP Server Architecture

Overview

Confidence: VERIFIED (source: `MCP_Server_Guide.md` from the EAP repo)

The AI Hub MCP capability is delivered via `iris-mcp-server` - a **standalone Rust binary** included in the IRIS `bin` directory. It acts as a protocol gateway between MCP clients (Claude Desktop, Claude Code) and IRIS backend tools.

Architecture



|
%AI.ToolMgr -> Your Tools/ToolSets

Transport Modes

Mode	Use Case	Configuration
stdio	Claude Desktop local	transport = "stdio"
HTTP	Remote MCP clients	transport = "http" , host/port
HTTPS	Production with TLS	transport = "https" + cert/key

Tool Registration Pattern

Tools are defined in ObjectScript as %AI.Tool or %AI.ToolSet subclasses, then exposed via an %AI.MCP.Service :

```
// 1. Define tools
Class SNOMED.Tools.Lookup Extends %AI.Tool
{
    /// Look up a SNOMED CT concept by its SCTID.
    ClassMethod GetConcept(sctid As %String) As %String
    {
        // Return JSON with concept details
    }
}

// 2. Compose into a ToolSet
Class SNOMED.ToolSet Extends %AI.ToolSet
{
    XData Definition [ MimeType = application/xml ]
    {
        <ToolSet Name="SNOMEDTools">
            <Description>SNOMED CT terminology operations</Description>
            <Include Class="SNOMED.Tools.Lookup"/>
            <Include Class="SNOMED.Tools.Search"/>
            <Include Class="SNOMED.Tools.Hierarchy"/>
            <Query Name="SearchConcepts" ... />
        </ToolSet>
    }
}
```

```
// 3. Create MCP Service
Class SNOMED.MCP.Service Extends %AI.MCP.Service
{
    Parameter SPECIFICATION As STRING = "SNOMED.ToolSet";
}
```

Then register in Management Portal: **System Administration > Security > Applications > MCP Servers** with path `/mcp/snomed`.

Configuration (config.toml for Claude Desktop)

```
[mcp]
transport = "stdio"

[[iris]]
name = "local"
server = { host = "localhost", port = 1972, username = "CSPSystem", password = "SYS"
}
pool = { min = 2, max = 5 }
endpoints = [
    { path = "/mcp/snomed" }
]

[logging]
level = "info"
output = "file"
file = "iris-mcp-snomed.log"
```

Claude Desktop config (`claude_desktop_config.json`):

```
{
  "mcpServers": {
    "snomed": {
      "command": "/path/to/iris-mcp-server",
      "args": ["--config=/path/to/config.toml", "run"]
    }
  }
}
```

Key Features

- **Service Discovery:** Auto-discovers `/mcp*` endpoints if `endpoints` omitted
- **Tool Refresh:** Background polling every 300s (configurable) with ETag-based conditional GET
- **Smart Discovery (RAG):** Semantic search over tool descriptions using local embeddings (AllMiniLML6V2)
- **Connection Pooling:** WebSocket session pool with configurable min/max
- **OAuth 2.1:** Supports bearer token passthrough for multi-tenant deployments
- **RBAC:** IRIS RBAC policies enforced per tool
- **Vault Integration:** HashiCorp Vault for secret management
- **Query-as-Tool:** SQL queries defined as class queries automatically become tools with typed parameters

Comparison with Current `sct` MCP Server

Feature	Current <code>sct</code> (Rust/stdio)	IRIS AI Hub
Transport	stdio only	stdio, HTTP, HTTPS
Tool definition	Rust code	ObjectScript XData XML
Auth	None	RBAC + OAuth 2.1
Scalability	Single process	Connection pool, multi-server
Monitoring	None	Real-time dashboard, OpenTelemetry
Smart discovery	No	RAG-based tool search

3. ObjectScript vs Python Trade-offs

How Embedded Python Works

Confidence: HIGH (verified against documentation structure + training knowledge)

- Python runs **in-process** with IRIS (shared memory space, no IPC)
- Each IRIS process has its own Python interpreter instance
- GIL is not a bottleneck because IRIS processes are single-threaded (parallelism via `Job`)
- `[ Language = python ]` keyword on methods enables Python in class definitions

Language Selection Guide for SNOMED Operations

Operation	Recommended	Rationale
RF2 file parsing	Python	<code>csv</code> module, better string handling, regex

Operation	Recommended	Rationale
Bulk loading (INSERT)	Python + SQL prepared statements	<code>iris.sql.prepare()</code> efficient for 800K+ records
Global-direct bulk load	Python <code>iris.gref</code>	Slightly slower than ObjectScript but simpler
Runtime concept lookup	ObjectScript	Zero marshaling overhead per call
Hierarchy traversal	ObjectScript	<code>\$ORDER</code> on globals is native and fastest
Transitive closure computation	Python	Graph algorithms (networkx if needed)
FTS/lexical search	SQL (either language)	IRIS SQL handles indexing
Vector similarity search	SQL (either language)	IRIS VECTOR type + SQL functions
MCP Tool definitions	ObjectScript	Required by <code>%AI.Tool</code> / <code>%AI.ToolSet</code> framework
External API calls	Python	<code>requests</code> , better HTTP libraries
NLP/embedding generation	Python	<code>sentence_transformers</code> , <code>ollama</code> client

Code Examples

Python class method for RF2 loading:

```
Class SNOMED.Loader Extends %RegisteredObject
{

ClassMethod LoadConcepts(filepath As %String) As %Status [ Language = python ]
{
    import iris
    import csv

    sql = iris.sql.prepare(
```

```

        "INSERT INTO SNOMED.Concept (ConceptId, Active, EffectiveTime, ModuleId,
DefinitionStatusId) "
        "VALUES (?, ?, ?, ?, ?)"
    )

    with open(filepath, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f, delimiter='\t')
        count = 0
        for row in reader:
            sql.execute(row['id'], int(row['active']), row['effectiveTime'],
                        row['moduleId'], row['definitionStatusId'])
            count += 1
            if count % 50000 == 0:
                print(f"  Loaded {count} concepts...")

        print(f"Total: {count} concepts loaded")
        return iris.cls("%SYSTEM.Status").OK()
    }

}

```

ObjectScript for hierarchy traversal:

```

ClassMethod IsDescendantOf(conceptId As %String, ancestorId As %String) As %Boolean
{
    // 0(1) check against transitive closure global
    Quit $DATA(^SNOMED.TC(ancestorId, conceptId))#2
}

ClassMethod GetDescendants(conceptId As %String) As %DynamicArray
{
    Set result = []
    Set desc = ""
    For {
        Set desc = $ORDER(^SNOMED.TC(conceptId, desc))
        Quit:desc=""
        Do result.%Push(desc)
    }
    Quit result
}

```

Performance Expectations

- Simple global SET: ObjectScript ~2-3x faster than Python `iris.gref`
- SQL INSERT via prepared statement: Roughly comparable (SQL layer dominates)
- Complex string parsing: Python often faster (better regex/string libs)
- Bulk `%Save()` : Similar (persistence layer dominates)

For 831K UK Monolith concepts: Python + SQL prepared statements should load in ~30-60 seconds on modern hardware. ObjectScript global-direct would be ~15-30 seconds.

4. Vector/Embedding Support

IRIS Native Vector Search

Confidence: HIGH (verified from iris-vector-search repo and docs references)

IRIS provides a native `VECTOR` SQL data type with: - **SIMD-accelerated** similarity functions - **ANN indexing** for datasets >100K vectors (significant performance improvement) - **Hybrid search**: combine vector similarity with standard SQL WHERE clauses

SQL Syntax (reconstructed from demos and docs)

```
-- Create table with vector column
CREATE TABLE SNOMED.ConceptEmbeddings (
  ConceptId VARCHAR(20) NOT NULL,
  Term VARCHAR(1024),
  Embedding VECTOR(DOUBLE, 384), -- dimension matches model output
  CONSTRAINT PK_ConceptId PRIMARY KEY (ConceptId)
)

-- Insert embedding
INSERT INTO SNOMED.ConceptEmbeddings (ConceptId, Term, Embedding)
VALUES (?, ?, TO_VECTOR(?))

-- Semantic search using cosine similarity
SELECT TOP 10 ConceptId, Term,
  VECTOR_COSINE(Embedding, TO_VECTOR(?)) AS Score
FROM SNOMED.ConceptEmbeddings
WHERE Score > 0.7
ORDER BY Score DESC

-- Hybrid search: semantic + active filter
```

```
SELECT TOP 10 ce.ConceptId, ce.Term,
            VECTOR_COSINE(ce.Embedding, T0_VECTOR(?)) AS Score
FROM SNOMED.ConceptEmbeddings ce
JOIN SNOMED.Concept c ON c.ConceptId = ce.ConceptId
WHERE c.Active = 1
ORDER BY Score DESC
```

ANN Index (for 831K+ concept embeddings)

```
-- Create ANN index for large datasets
CREATE INDEX IX_Embedding ON SNOMED.ConceptEmbeddings(Embedding)
TYPE VECTOR WITH OPTIONS '{"type": "HNSW", "m": 16, "efConstruction": 200}'
```

Integration with AI Hub

The AI Hub’s **langchain SDK** (`langchain-intersystems` package) provides: - `VectorStore` implementation backed by IRIS - Integration with Config Store for credential management - LLM and MCP configurations stored centrally

```
# From langchain SDK demo
import iris
from langchain_intersystems.chat_models import init_chat_model
from langchain_intersystems import init_mcp_client

conn = iris.connect('localhost', 51774, 'USER', '_SYSTEM', 'SYS')
model = init_chat_model('openai', conn) # pulls config from Config Store
```

Embedding Generation Strategy

For your SNOMED port, two options:

Approach	Pros	Cons
Ollama (local)	Free, private, matches current sct approach	Slower, requires Ollama running
OpenAI/cloud	Faster batch embedding	Cost, data leaves premises

Recommended: Use Ollama via the AI Hub’s OpenAI-compatible provider config:

```
Set provider = ##class(%AI.Provider).Create("openai", {
    "base_url": "http://localhost:11434/v1/",
    "api_key": "ollama"
})
```

Embed concepts in batches using Python, store in IRIS VECTOR columns, expose semantic search as an MCP tool.

5. Packaging & Distribution Model

IPM (InterSystems Package Manager)

Confidence: HIGH (verified from IPM repo docs)

IPM is the standard distribution mechanism for IRIS applications:

Aspect	Detail
Config file	<code>module.xml</code> in package root
Registry	pm.community.intersystems.com (public) or private ORAS
Scope	Per-namespace installation
Python deps	Supported via <code>requirements.txt</code> + installer hook
IRIS version	IPM 0.10.x requires IRIS >2022.1

Package Structure for SNOMED Tool

```
snomed-iris/
  module.xml
  src/
    SNOMED/
      Concept.cls          -- %Persistent class for concepts
      Description.cls       -- %Persistent for descriptions
      Relationship.cls       -- %Persistent for relationships
      Loader.cls            -- RF2 loading (Python methods)
      TransitiveClosure.cls -- TC computation
    Tools/
      Lookup.cls           -- %AI.Tool for concept lookup
      Search.cls           -- %AI.Tool for lexical search
```

```

Hierarchy.cls      -- %AI.Tool for hierarchy queries
Semantic.cls       -- %AI.Tool for vector search
ToolSet.cls        -- %AI.ToolSet composing all tools
MCP/
  Service.cls      -- %AI.MCP.Service
config/
  mcp-config.toml  -- iris-mcp-server configuration template
data/
  requirements.txt  -- Python dependencies
README.md

```

module.xml Example

```

<?xml version="1.0" encoding="UTF-8"?>
<Export generator="IRIS" version="26">
  <Module>
    <Name>snomed-iris</Name>
    <Version>1.0.0</Version>
    <Description>SNOMED CT Terminology Services for IRIS AI Hub</Description>
    <Keywords>snomed,terminology,fhir,mcp,ai-hub</Keywords>
    <SourcesRoot>src</SourcesRoot>
    <Resource Name="SNOMED.PKG"/>
    <Dependencies>
      <Module Name="iris-fhir-server" Version=">=2.0.0"/>
    </Dependencies>
    <Invoke Class="SNOMED.Installer" Method="Setup" Phase="Configure"/>
  </Module>
</Export>

```

Container Distribution

For Docker-based deployment (matching your current setup):

```

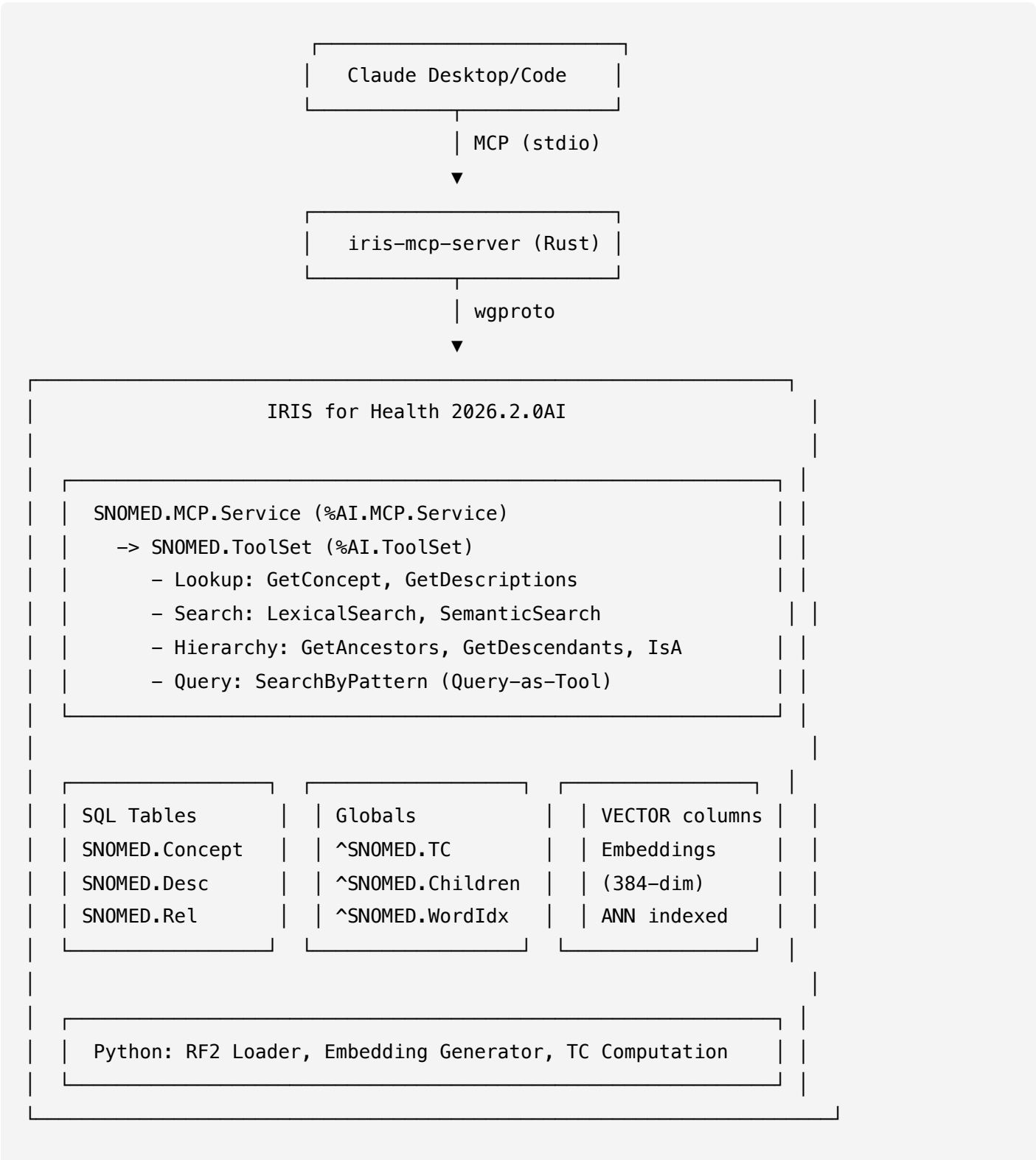
FROM containers.intersystems.com/intersystems/irishealth-community:2026.2.0AI
# Install IPM package
RUN iris start IRIS && \
    iris session IRIS -U USER "zpm \"install snomed-iris\" \" && \
    iris stop IRIS quietly
# Copy MCP server config

```

```
COPY config/mcp-config.toml /opt/iris/config/
EXPOSE 1972 52773
```

6. Recommended Architecture

High-Level Design



|
▼ (optional)

Ollama (embeddings)

Feature Mapping: sct (Rust) -> IRIS AI Hub

Current sct Feature	IRIS Implementation	Notes
RF2 -> NDJSON	Python [Language = python] loader -> SQL	Bypass NDJSON; load directly into IRIS
SQLite + FTS5	IRIS SQL + iFind/BM25 text index	Native full-text search built into IRIS
Parquet export	Python pyarrow or SQL export	Can export from IRIS tables
MCP server (stdio)	iris-mcp-server + %AI.MCP.Service	More capable: HTTP, auth, monitoring
Vector embeddings	IRIS VECTOR type + SQL similarity	Native, SIMD-accelerated
Semantic search	VECTOR_COSINE() in SQL	ANN indexed for 831K concepts
Transitive closure	Globals: ^SNOMED.TC(ancestor,desc)	O(1) lookup, computed at load time

7. Key Risks & Mitigations

Risk	Impact	Mitigation
EAP APIs may change before GA	Medium	Isolate AI Hub dependencies; use adapter pattern
No SNOMED-specific FHIR support built-in	Low	You're building this - it's the value proposition
Python GIL for concurrent embedding	Low	IRIS Job model sidesteps GIL (separate processes)
ANN index behavior for 831K vectors	Low	Well within documented performance range (>100K)
<code>iris-mcp-server</code> is new/pre-release	Medium	Rust binary is standalone; can fall back to direct API

8. Recommended Next Steps

- 1. Prototype the data model:** Create `SNOMED.Concept` , `SNOMED.Description` , `SNOMED.Relationship` persistent classes in your Docker container
- 2. Build RF2 loader:** Python method using `iris.sql.prepare()` for bulk insert
- 3. Implement transitive closure:** Compute and store in `^SNOMED.TC` global
- 4. Create first MCP tool:** Simple concept lookup via `%AI.Tool`
- 5. Test with Claude Desktop:** Configure `iris-mcp-server` + `config.toml`
- 6. Add vector search:** Generate embeddings for description terms, store in VECTOR column
- 7. Package as IPM module:** Create `module.xml` for distribution

Sources

Source	Type	Confidence
AI Hub EAP - MCP Server Guide	Primary documentation	Verified
AI Hub EAP - ObjectScript SDK Guide	Primary documentation	Verified
AI Hub EAP - LangChain SDK	Primary documentation	Verified
AI Hub EAP - Config Store Guide	Primary documentation	Verified

Source	Type	Confidence
IRIS Vector Search repo	Community example	Verified
IPM repo	Official tool	Verified
InterSystems IRIS Embedded Python docs	Official docs	Structure verified; content from training data
IRIS for Health terminology services	Official docs	Navigation-only access; content from training data

Research completed: 2026-05-13